Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Dersin tarihi 4 Kasım 2020 Çarşamba. Bu kayıt, ders 9'dan sonraki uygulama saatidir. Zoom üzerinden yapıldı.
  - Bu hafta uygulama yapılmadı. Konu anlatımına devam edildi.
- Merge sort: özyinelemeli sözde kod, Merge fonksiyonu, en kötü durum rekürans bağıntısı ve Master teoremiyle çözümü.
- Quick sort: çalışma mantığı, pivot, Hoare bölümlemesi (Hoare partitioning) ve en iyi durum analizi.
- Gelecek haftanın planı: quick sort'un en kötü ve ortalama durumu, bir uygulama sorusu, ardından hashing.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu dersin amacı merge sort'u öğretmek değil. Amaç, rekürans bağıntısını yazabilmek ve algoritmanın büyüme derecesini hesaplayabilmek.
  - Merge sort ve quick sort'un ayrıntıları Veri Yapıları dersinde anlatılmıştı.
- Merge sort'ta bölerken iş yapılmaz, asıl iş birleştirmede (merge) yapılır. Quick sort'ta bunun tersi geçerlidir: asıl iş bölümlemede (partitioning) yapılır, birleştirme kolaydır.
- Rekürans bağıntısı önce yazılır. Sonra ister geriye doğru yerine koyma (backward substitution), ister Master teoremi ile çözülür.
- Master teoreminde a, kaç alt problemin çözülmeye devam edildiğidir.
  - Quick sort'ta parçalar eşit olmak zorunda değildir. Ama iki parça da çözülmeye devam edildiği için $a = 2$'dir. İkili aramada $a = 1$'dir.
- Merge sort'un en kötü durumu ile merge işleminin en kötü durumu farklı şeylerdir. Burada merge işleminin en kötü durumu hesaplanıyor.
- Merge sort, karşılaştırma sayısı bakımından teorik alt sınıra en yakın sıralama algoritmasıdır.

Kısa Tekrar Notları

- Merge sort en kötü durum:

$$C_w(n) = 2C_w(n/2) + C_{merge}(n), \quad C_{merge}(n) = n - 1, \quad C_w(1) = 0$$

- Master teoremiyle: $a = 2$, $b = 2$, $d = 1$. $a = b^d$ olduğu için $C_w(n) \in \Theta(n \log n)$.
- Kesin çözüm ($n = 2^k$): $C_w(n) = n \log_2 n - n + 1$.
- İki sıralı yarıyı birleştirirken en kötü durumda $n - 1$, en iyi durumda $n/2$ karşılaştırma yapılır.
- Quick sort en iyi durum: $C_b(n) = 2C_b(n/2) + n$, $C_b(1) = 0$. Sonuç $\Theta(n \log n)$.
- Hoare bölümlemesinde pivot ilk eleman: $p = A[l]$.
  - i soldan, $A[i] \geq p$ olana kadar ilerler.
  - j sağdan, $A[j] \leq p$ olana kadar ilerler.
  - $i < j$ ise $A[i]$ ile $A[j]$ yer değiştirir. $i \geq j$ olunca durulur.
  - Sonunda pivot $A[j]$'nin yerine konur.
- Master teoremi $T(n) = aT(n/b) + f(n)$ biçimindeki bağıntılara uygulanır.

Detaylı Açıklamalar

Merge sort sözde kodu

```
Mergesort(A[0..n-1])
  if n > 1
    copy A[0..floor(n/2)-1] to B[0..floor(n/2)-1]
    copy A[floor(n/2)..n-1] to C[0..ceil(n/2)-1]
    Mergesort(B[0..floor(n/2)-1])
    Mergesort(C[0..ceil(n/2)-1])
    Merge(B, C, A)
```

```
Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])
  i <- 0; j <- 0; k <- 0
  while i < p and j < q do
    if B[i] <= C[j]
      A[k] <- B[i]; i <- i + 1
    else
      A[k] <- C[j]; j <- j + 1
    k <- k + 1
  if i = p
    copy C[j..q-1] to A[k..p+q-1]
  else
    copy B[i..p-1] to A[k..p+q-1]
```

- Durma koşulu $n = 1$'dir. Tek elemanlı dizi zaten sıralıdır.
- Temel işlem, Merge içindeki `B[i] <= C[j]` karşılaştırmasıdır.

Merge işleminin en kötü ve en iyi durumu

- En kötü durum: elemanlar iki yarıdan sırayla gelir. Örnek: $[2, 4, 6, 8]$ ile $[1, 3, 5, 7]$.
  - 1-2, 2-3, 3-4, 4-5, 5-6, 6-7, 7-8 karşılaştırmaları yapılır. Toplam $7 = n-1$ karşılaştırma.
- En iyi durum: bir yarının bütün elemanları diğerininkinden küçüktür. Örnek: $[1, 2, 3, 4]$ ile $[5, 10, 100, \dots]$.
  - Küçük yarı bitince diğer yarının kalanı karşılaştırmasız kopyalanır. Yaklaşık $n/2$ karşılaştırma.
- Sıralama algoritmalarında en kötü durum genellikle ters sıralı dizidir. Ama burada konuşulan, merge işleminin en kötü durumudur.

Merge sort analizi

- Rekürans bağıntısı:
  - $2C_w(n/2)$: Dizi ikiye bölünür ($b = 2$) ve iki yarı da sıralanır ($a = 2$).
  - $n - 1$: Birleştirmede yapılan karşılaştırmalar. $f(n) = n - 1 \in \Theta(n)$, yani $d = 1$.
  - Başlangıç koşulu: $C_w(1) = 0$. Tek eleman kalınca karşılaştırma yapılmaz.
- Master teoremi: $a = 2 = b^d = 2^1$. İkinci durum: $C_w(n) \in \Theta(n^d \log n) = \Theta(n \log n)$.

Quick sort ve Hoare bölümlemesi

- Bir pivot seçilir. Pivottan küçükler sola, büyükler sağa yerleştirilir. Pivot doğru yerine gider. Sonra iki taraf ayrı ayrı sıralanır.
- Bölme ortadan değil, pivotun yerleştiği konumdan (s) yapılır.

```
Quicksort(A[l..r])
  if l < r
    s <- HoarePartition(A[l..r])
    Quicksort(A[l..s-1])
    Quicksort(A[s+1..r])
```

```
HoarePartition(A[l..r])
  p <- A[l]
  i <- l; j <- r + 1
```

```
repeat
  repeat i <- i + 1 until A[i] >= p
  repeat j <- j - 1 until A[j] <= p
  swap(A[i], A[j])
until i >= j
swap(A[i], A[j])        // son (fazladan) yer değiştirmeyi geri al
swap(A[l], A[j])        // pivotu yerine koy
return j
```

- Derste örnek: pivot 5. Soldan ilk büyük eleman 9, sağdan ilk küçük eleman 4. Bunlar yer değiştirir. Sonra 8 ile 2 yer değiştirir. İndisler kesişince pivot ortaya yerleşir.
- Pivot burada ilk eleman seçildi. Algoritmanın verimini artırmak için başka pivot seçimleri de vardır.

Quick sort en iyi durum

- En iyi durumda her bölümleme diziyi tam ortadan iki eşit parçaya ayırır.
- Bölümlemede i ve j dizinin her iki ucundan ilerler. Toplamda yaklaşık n karşılaştırma yapılır. $f(n) = n$.
- $C_b(n) = 2C_b(n/2) + n$, $C_b(1) = 0$. Master teoremi: $a = 2$, $b = 2$, $d = 1$, $a = b^d$. Sonuç $\Theta(n \log n)$.
- En kötü ve ortalama durum bir sonraki hafta işlenecek.

Ders sonrası notlar

- **Hoare bölümlemesinde eksik satır:** Tahtadaki sözde kodda döngüden sonra bir `swap(A[i], A[j])` satırı var. Bunun işlevi derste söylenmedi. İç `repeat` döngüsü $i \geq j$ olduktan sonra da bir yer değiştirme yapar. Bu satır o son, hatalı yer değiştirmeyi geri alır. Sonra pivot `swap(A[l], A[j])` ile yerine konur.
  - Ayrıca i dizinin sonunu geçebilir. Bunun için büyük olasılıkla kitaptaki gibi dizinin sonuna pivottan büyük bir gözcü (sentinel) eleman konur.
- **Sıralı dizi en iyi durum değildir:** Bir öğrenci "en iyi durumda dizi zaten sıralıdır" dedi. Hoca bunu kabul etmedi ve haklıydı. Pivot ilk eleman seçildiğinde sıralı dizi quick sort'un en kötü durumudur. Her bölümleme bir tarafı boş bırakır ve $\Theta(n^2)$ karşılaştırma yapılır. En iyi durum, pivotun her seferinde ortanca olmasıdır.
- **Karşılaştırma sayısı:** Kitaptaki analize göre büyük olasılıkla bölümlemede indisler kesişirse $n + 1$, çakışırsa n karşılaştırma yapılır. Büyüme derecesi bakımından $f(n) \in \Theta(n)$ olduğu için sonuç değişmez.
- **Master teoremi yalnız böl ve yönet için mi?** Hoca "evet, yalnız böl ve yönet bağıntıları" dedi. Teorem, $T(n) = aT(n/b) + f(n)$ biçimindeki her bağıntıya uygulanır. İkili arama gibi sabit oranda azaltma algoritmaları da bu biçimdedir ($a = 1$). Derste ikili arama da zaten böyle çözüldü.
- **Merge sort kesin değer:** $n = 2^k$ için $C_w(n) = n \log_2 n - n + 1$. Python ile $n = 2, 4, 8, 16, 32$ için 1, 5, 17, 49, 129 elde edildi. Formülle aynıdır.
- **Teorik alt sınır:** Karşılaştırmaya dayalı sıralamada en kötü durumda en az $\lceil \log_2 n! \rceil \approx n \log_2 n - 1{,}44n$ karşılaştırma gerekir. Merge sort'un en kötü durumu bu sınıra yakındır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.